



UNIVERSIDAD DE LAS FUERZAS ARMADAS - ESPE

DATE:

17/05/2026

CLASS:

AWD - 30716

HOMEWORK:

Project Review

Team 1: ByteClub, Leader Nelson Andino

Project: Biconoirs Restaurant

Inspector: Ricardo Monge

GitHub: <https://github.com/MagZerock/AWD-30716-BYTECLUB.git>

Database URL:

<https://supabase.com/dashboard/project/teqgmzwpccwcelrpecdyq/editor/17911?schema=public>

Web App URL: <https://prototype-tv4q.onrender.com>

01) DYLAN ALEXANDER ALVARADO PALACIOS

02) NELSON DAVID ANDINO MARTINEZ

03) JILMAR JHOEL CALDERON CALVA

Ord.	Artifact	Evaluation	Observations
7	PB	9.7/10	One task mentions setting up the base template in Spanish, however, there are small parts in English. - Some images are not loading. The button to log out and access the administrator panel disappears instantly. 9 Futures + 1 - 0.3
8	UCD	9.6/10	-The use case is very general: reserve a table or view an active reservation. There is no management or handling within the administrator panel. -There is a login, but that is not a use case. -Task 3.4 is not described in the use case diagram; there is no order management (completed or canceled). -Task 3.6 is not included in the diagram; there is literally no interaction with uploading a survey.
9	CD	9.7/10	- The diagram shows inheritance, but there is no user inheritance for customer or administrator because roles are used in the code. - The diagram shows a MenuItem class, but it's called Dish in the code. - There is no deductQuantify() in InventoryBatch

10	Execution	37/40	Despite minor user experience issues, the system fulfills its purpose. However, there are inconsistencies in the web service that should be detailed in the Product Backlog as a business rule.
11	Clean Code	35/40	- Use of generic variables like \$data - Use of prefixes, for example, \$tempDetail could be called \$simulatedItemDetail - Functions should be small and understandable, emphasizing single responsibility. Example: the reservations() function - Use of comments // sync ingredients - The CD format is not maintained according to Dish.php; instead, MenuItem is used
12	MVC	18/20	Despite the well-organized MVC directory structure, indicating the use of frameworks, important files are not visible; for example, HTML files are not displayed in the view. The entire system relies on the PHP language.

Team 2: CodeXplorers, Leader Andrés Cardenas.

Project: Dental Fable

Inspector: HTML-Team (Team 3)

GitHub: <https://github.com/andrescardenase77-web/AWD-30716-CodeXplorers.git>

Database URL:

Web App URL: <https://fabula-dental-managment.onrender.com/>

- Administrador: Usuario: admin | Clave: Admin12!
- Odontólogo: Usuario: dentist | Clave: Denti12!
- Recepcionista: Usuario: reception | Clave: Recep12!

04) ANDRES SEBASTIAN CARDENAS ERAZO

05) KERLY ANDREINA CHUQUI AGUINDA

06) CAMERON VICTORIA DIAZ CAIZALUISA

Ord.	Artifact	Evaluation	Observations
7	PB	9.7 / 10	- Syntax: Most of the backlog items use noun phrases instead of the required verb + complement syntax like "New patient registration" instead of "Register new patient". - Feature structure (CRUD): CRUD

			operations are separated into individual backlog items, especially into PB-06 Editing and PB-07 Removal, instead of being consolidated under a main entity feature, contradicting the core feature. - Unapproved Roles: The Backlog introduces a "Visitor" role into PB-17 and a "Clinic visitor" instead into US-08, that were not defined which strictly established only 3 roles (Dentist, Administrator, Receptionist). Additionally, there are translation professionalism errors in the status column like "Earring" not "Pending".
8	UCD	9.6 / 10	- Traceability Flaws: There is a notable discrepancy between the Product Backlog and the UCD. Critical functionalities prioritized in the backlog, such as 'Edit patient' into the PB-06 and 'Delete patient' of the PB-07, are entirely absent from the Dentist's use cases. The diagram must accurately reflect all features committed in the PB. - Omission of System Actors: The diagram fails to model the 'Visitor' actor, which was explicitly defined in both the User Stories number US-08 and the Product Backlog defined in PB-17 for accessing the public treatments page. All interacting entities must be represented. - Architectural Misconception: The 'Login' process is erroneously modeled as a primary business use case directly connected to all main actors. Following standard UML design principles, authentication should be treated as a system precondition or a security constraint rather than a standalone functional use case. - Typographical Errors: The diagram contains spelling mistakes in core elements, such as labeling the receptionist actor as 'Rcepcionist', which denotes a lack in submission.
9	CD	9.6 / 10	- Redundant Role Modeling: The

			diagram implements both an inheritance hierarchy into Receptionist, Dentist, Administrator inheriting from User, and a role attribute within the User parent class. This is architecturally redundant because the standard practice dictates using either specialization like “subclasses” or a “role” attribute, but not both simultaneously. - Missing Structural Associations: There are no association lines connecting the Dentist class to the Patient class, nor the Receptionist class to the Payment class. These are divided structural relationships that must be represented visually. - Typographical Errors: The Supply class contains attribute naming typos, specifically unitCoast instead of unitCost and expirationData instead of expirationDate. - Incomplete CRUD Methods: As a disclaimer, the classes do not explicitly model all standard CRUD operations that were "included" in their methods like the Dentist class lacks update and delete methods for Patient objects.
10	Execution	33 / 40	- Logical flaws in core features: The system allows registering patients with illogical dates of birth like 0 years old or future dates, failing to block the transaction and only requesting a legal guardian with in-client validations. - Financial Integrity discrepancies: Payments can be processed for unregistered patient IDs, creating orphaned records. Additionally, selecting "Final Payment" automatically sets the status to "Completed" even for negligible amounts like \$0.01\$ breaking the intended financial logic. - Time/Date validation errors:

			The payment module incorrectly blocks past dates while displaying an error message meant for future dates. In the inventory, items expiring on the exact current date are ambiguously flagged as "Next to Expire" instead of "Expired" without time validation. - SPA/Architecture violations: Despite the non-functional requirements in RNF-03, inventory CRUD operations force full page redirects instead of asynchronous DOM updates without reloading. - Poor UX and Frontend Dependency: The system relies heavily on basic HTML5 attributes for data integrity, displaying unhandled browser-default warnings in English like "Please enter a number", degrading the professional user experience or also spanish JS warnings into the same space.
11	Clean Code	31 / 40	- Security & Hardcoding: Database credentials are exposed in plain text within dbCredentials.php and api_patients.php instead of using environment variables. - Dead Code: The calculateStatus() function in the Supply class contains unreachable code blocks regarding empty expiration dates, which are already filtered in previous methods. - Opaque Exception Handling: Controllers catch general throwables "(catch (Throwable \$e))" but silently redirect to error pages without logging the actual stack trace or message, making backend debug impossible. - Method Signature Mismatch: The Receptionist class diagram

			dictates a plural registerPayments method, which contradicts both its singular object parameter and the singular transaction logic defined in the backlog. - Unsafe Data Instantiation: Age calculation relies on instantiating new “DateTime(\$patient->birthday)” without prior strict string validation, exposing the API to fatal PHP exceptions if corrupted data bypasses the frontend to the backend.
12	MVC	19 / 20	- Solid Separation: The project demonstrates a well-organized and explicit MVC directory structure. System responsibilities are clearly segregated into models, controllers, and views folders. - Strict Asset Categorization: The views directory correctly sub-categorizes assets into html, css, js, and php subfolders, while the models folder strictly contains domain data classes without mixing any presentation layers like CSS or HTML. - Root Exposure Vulnerability: The main entry point form index.php and sensitive configuration files like dbCredentials.php are placed directly at the project root. In a robust MVC architecture, the entry point should ideally be isolated within a public/ directory to prevent direct web access to the controllers directories.

Team 3

Project: HTML-Team, Leader César Galarza.

GitHub: Ser-salud Phisioterapy Clinic

Inspector: T6

GitHub: <https://github.com/DiverCesar/AWD30716-HTML.git>

Database URL:

<https://console.neon.tech/app/projects/crimson-bird-22689774/branches/br-patient-dust-anvlnbs/tables?database=neondb>

Web App URL: <https://flova-front.vercel.app/login>

07) LENIN ISAAC ERAZO CAMPOS

08) CÉSAR LUIS GALARZA VILLALVA

09) BRAYAN PATRICIO GUALOTUÑA AMAGUAYA

Ord.	Artifact	Evaluation	Observations
7	PB	10 /10	they organize the features by Epic, each one has the established number of tasks and syntaxis
8	UCD	9.3 /10	The diagram shows includes, but not extends system is not an actor for an use case diagram, wrong name choice names consistency
9	CD	9.2 /10	presents some UML syntax inconsistencies missing multiplicity relationships are not well represented
10	Execution	30/40	its mandatory to add a motive for an appointment cancellation but not a restriction to make sure the patient actually writes something when a therapy was deactivated it disappeared and the counter for inactive therapies did not change, it could not be found to be activated again could not find the option to confirm an appointment admin has not access to view the appointments did not provide a specialist/therapist account to review their portal
11	Clean Code	25/40	unnecessary comments, noise Spanish-English mixing Massive functions
12	MVC	15/20	The project has a good organization but not an explicit MVC architecture

			its divided in front end and back end, has a folder called controllers, but not one called models, in the front end folder exist a section called pages that can be interpreted as the view
--	--	--	---

Team 4: SharkHub Ricardo Monge

Project: Panda Black And White Barbershoop

Inspector: T2, Andrés Cardenas.

GitHub: <https://github.com/Monge-Ricardo/AWD30716-SHARKHUB.git>

Database URL: email

Web App URL: <http://3.141.169.122>

10) GABRIEL ANTHONY MOLINA GALLEGOS

11) RICARDO ANDRES MONGE MIÑO

12) ALEJANDRO XAVIER OBANDO ZAPATA

Ord.	Artifact	Evaluation	Observations
7	PB	9.8 /10	7 features + 1
8	UCD	9.8 /10	System usually is not an actor, there are syntax error
9	CD	9.9/10	Method “operation()” in class BarberShop is ambiguous
10	Execution	35/40	When registering a customer, the data changes, the appointment scheduling doesn't work, the barber can't attend an appointment, the system doesn't show error messages when filling out forms with incorrect parameters, and there's an inconsistency between the services the customer sees and those the barber sees.
11	Clean Code	36/40	There are comments in the code, abstract class Controller is empty, styles in line in some files, there are javascript in html (onclick) in some files
12	MVC	17/20	The MVC application is hidden inside the app folder and it's confusing. There's no JavaScript in the models folder and no CSS in the views folder; they are inside the public folder.

Team 5 DEA, Esteban Quiroga

Project: Accounting System

Inspector: T1, ByteClub

GitHub: <https://github.com/EstebanQuiroga17/ESPE2604-AWD30716-DEA.git>

Database URL: <https://tmjxgxiwmqfvavjnbvj.supabase.co>

Web App URL: <https://ats-app-t6f3.onrender.com>

13) ESTEBAN ARIEL QUIROGA BENITEZ

14) DAVID JOSUE RODRIGUEZ VILLARROEL

15) ANGEL JEANPIERRE SABANDO MORA

Ord.	Artifact	Evaluation	Observations
7	PB	9.7/10	- Ambiguity in FT-10 "Dashboard". The ID FT-10 appears only as "Dashboard" without detailed effort and lacks clarity in the summary table (the "hrs" appears misaligned). A name should be meaningful; "Dashboard" is too generic. It should be something like a SalesPerformanceDashboard or FiscalSummaryDashboard. - Incomplete technical task T-02-01. In the description of task T-02-01 (SRI Handshake), the text is truncated: "Connect the system with the...". Documentation, like code, should be self-explanatory and not leave "comments" or descriptions for the media. - Diffuse responsibility in FT-06. Assigning the same feature to A.S./D.R. without dividing who does what violates the Single Responsibility principle. It is preferable to assign a task to a single person or divide the feature into individual technical tasks with a single responsible party for each. They have 10 features - 0.3
8	UCD	9.8/10	-In the diagram, UC-02: Logging into the platform is not a use case, but a precondition or security constraint. -An actor is missing in the use cases to handle UC09, since the characteristics designate the Auditor/Admin.

9	CD	9.7/10	-ProcessTracker stores attributes such as "steps" that track the state, whether the object is composed of others or has a permanent structural relationship, it must be an association. -In the class diagram, only Taxpayer and Accountant inherit from the User class. The role of Auditor or Administrator according to US-09 is missing. -The Product Backlog defines Feature FT-02 as "Creation of isolated local directories based on specific taxpayer RUCs and periods" The diagram does not include any class representing this "Workspace" or physical directory manager.
10	Execution	33/40	Although it says it should allow invoice downloads, it doesn't. It only displays a notification that the download is in progress, but no file actually downloads. The ATS XLSM file says it was generated, but it doesn't allow the file to be downloaded. When editing users, it doesn't clearly indicate the error; it only displays a "Path not found" message without specifying whether the problem is with the username or email address. It doesn't allow editing users even though all fields are correctly filled out. It doesn't allow deleting users; they remain in the system. The only message displayed is "Path not found," without specifying the specific problem. It doesn't allow uploading files, even with the required extension; it works with overwritten data.
11	Clean Code	35/40	Duplication (DRY): Multiple identical functions (e.g., generateUuid()) repeated in controllers/repositories instead of a common helper. Tight coupling: Classes instantiate

			dependencies directly (e.g., new User()/User::create() in controllers) instead of using injection or unified repositories. Poor/inconsistent naming: Ambiguous identifiers (findById vs. find, createAt vs. create_at) hinder readability and maintenance. Magic numbers/strings: Hardcoded values (e.g., '#TKT-2025-0047', ports, lengths) instead of named constants or configurations. Overly long functions/methods: Methods in controllers that perform multiple steps (parse input, validate, create, format response) should be split. Lack of structured logging: Captures that return simple messages without logging context for traceability and debugging.
12	MVC	19/20	It correctly applies the MVC pattern by separating responsibilities between controllers and models; however, the project root should not contain src folders or loose Vite/React configuration files. Only the ATSWebProjectFE and BackProjectWebPHP folders should be present.

Team 6

Project: American Latin Class, Carlos Torres

Inspector: T5, Esteban Quiroga

GitHub: <https://github.com/EvelynVillarreal/AWD-30716-Code-Coffee>

Database URL:

<https://supabase.com/dashboard/project/luzlnndzhpilgxacnim/settings/general?method=github>

Web App URL: <https://creative-pothos-6c7a4c.netlify.app/>

Datos De Prueba:

- alumno@americanlatinclass.com
- director@americanlatinclass.com
- profesor@americanlatinclass.com
 - contraseña: ALC2026*

16) CARLOS ALEXANDER TORRES PINCAY

17) EVELYN HAYDEE VILLARREAL SALGADO

Ord.	Artifact	Evaluation	Observations
	PB	9.5/10	The backlog is well structured with 3 features, 9 tasks in Jira, and 20 items correctly prioritized and traced to their requirements. A deduction of -0.5 applies because sub-items PB-006A and PB-006B were added outside the scope agreed upon in the meeting without formal customer validation, and PB-020 remains without an assigned requirement. 3 features + 2 - 0.5
	UCD	9.2/10	-0.4 There are no <<include>> or <<extend>> relationships defined anywhere in the diagram, neither in the PUML source nor in the drawio version. This is the most significant UML structural omission. -0.4 The General Director and Administrator share identical access to UC10, UC11, and UC12 with no role differentiation shown in the diagram. Additionally, UC6 (Register teacher attendance) is assigned exclusively to the Branch Director, omitting the Teacher or Administrator as valid actors for that use case.
	CD	9.3/10	The class diagram covers all 10 domain classes and supports every use case defined in the UCD. Multiplicities are present but use the word "many" instead of the UML standard * notation. There is no inheritance hierarchy: the User class stores role as a plain string instead of modeling Student, Teacher, and Administrator as specializations. All relationships are plain associations with no distinction between composition

			and aggregation, which does not reflect the actual lifecycle dependencies between classes such as Branch → BranchFinanceReport or ProfessionalEvent → DancerEventAssignment.
	Execution	28/40	The portal presents a functional foundation with the main modules implemented (enrollment, attendance, finance, and B2 Professional). However, 3 functional or security errors and 5 necessary improvements were identified. The most critical flaws are: the password field displaying input as plain text, a +1 day offset across all automatic date fields, the absence of real data validation in the attendance and kiosk modules, and the registration of duplicate events without any alert or warning. It is recommended to correct the critical errors before using the portal in production with real data.
	Clean Code	31/40	Code principles reveals code that is functionally correct but structurally deficient in several key areas. Regarding naming, most classes and methods are clear and intention-revealing; however, magic strings such as "teacher", "Pending", the numeric value 7000, and hardcoded locale strings are used throughout, and the API base URL is defined directly inside the constructor rather than as an environment variable. In terms of function design, the request() method in ApiClient carries five distinct responsibilities, bindEventForm() spans approximately 40 lines performing six different tasks, and loadData() mixes data loading for all roles within a single conditional block, violating the

			single-responsibility principle. The most critical violation lies in class design: DashboardController acts as a God Class by simultaneously managing session validation, navigation, data loading, rendering of six views, and binding of five forms, when it should be split into independent components such as SessionGuard, NavigationController, DataLoader, and role-specific views. While the utility classes Dom and Formatters are reused consistently following the DRY principle, the construction of HTML tables through template literals is repeated across multiple methods without a shared abstraction. On error handling, the absence of a timeout in ApiClient, a silent fallback with hardcoded branches in BranchStore, and an unhandled edge case in firstError() were identified as weaknesses. Coupling is another critical concern, as fillSelects() and DateDefaults directly access hardcoded HTML element IDs, and findSelectedDancer() performs a fragile string comparison instead of matching by student ID. Finally, the file is entirely lacking in comments, JSDoc, or any documentation of its flow and inter-class dependencies, and no unit or integration tests exist for any of its methods or classes.
	MVC	17/20	The project follows an MVC pattern distributed across three main folders within 06Code/: View for the frontend, Model for data models, and Controller for the backend. The View layer is correctly composed of HTML, CSS, and JavaScript files organized into css/ and script/ subfolders, though it contains

			non-presentation files such as netlify.toml and README.md that should reside at the project root; furthermore, all JavaScript logic is concentrated in a single monolithic app.js file rather than being distributed into independent modules by functionality. The Controller layer is the best-structured part of the project: it organizes the PHP backend modularly with clearly delimited subfolders for controllers, services, validators, middleware, support utilities, and routes, following good separation-of-concerns practices. However, naming the folder Controller is imprecise since it contains the entire backend, not just controllers. The Model layer, while containing ten correctly named domain models with a shared BaseModel.php base class, presents the most serious structural issue: it is located as a sibling folder of the backend (06Code/Model/) rather than inside the PHP project (Controller/src/Model/), which prevents Composer's autoloader from referencing it correctly and breaks the actual integration between the model and the backend.
--	--	--	---